CHANDIGARH UNIVERSITY

OBJECT-ORIENTED PROGRAMMING PROJECT

EMPLOYEE PAYROLL MANAGEMENT SYSTEM

DESIGNED AND IMPLEMENTED USING C++

A MENU-DRIVEN OBJECT-ORIENTED APPLICATION

SUBMITTED BY	ACADEMIC DETAILS
Manav Jit Singh UID: 25BCD10004 Section: 25BCD1(A)	Subject: **Object-Oriented Programming (C++)** Subject Code: **25CAH-201** Programme: BCA (Hons.) with Research in Data Science
SUBMITTED TO	**ACADEMIC SESSION**
Teacher Name: **Suman**	2026–2027

PROJECT REPORT

CERTIFICATE

This is to certify that **Manav Jit Singh**, UID **25BCD10004**, Section **25BCD1(A)**, has completed the project titled **"Employee Payroll Management System using C++"** for Object-Oriented Programming, Subject Code **25CAH-201**, under the guidance of **Ms. Suman**.

The project demonstrates the practical use of classes, objects, encapsulation, inheritance, abstraction, runtime polymorphism, constructors, virtual functions, dynamic memory management and file handling in C++.

Faculty Signature	Date	Department Seal
_____	_____	_____

DECLARATION

I, **Manav Jit Singh**, declare that this project report entitled **"Employee Payroll Management System using C++"** has been prepared for academic purposes as part of the Object-Oriented Programming subject. The design, explanations and program are presented for learning and demonstration.

I understand the logic and concepts described in this report and acknowledge the references used for learning C++ programming.

Student Signature: _____ **Date:** _____

ACKNOWLEDGEMENT

I express my sincere gratitude to **Ms. Suman** for guidance and support during the preparation of this project. I am thankful to Chandigarh University for providing the academic environment and learning resources that helped me understand object-oriented programming and its practical applications.

I also acknowledge the C++ documentation and learning resources consulted while designing the classes, salary calculations, menu operations and file-storage approach.

ABSTRACT

The Employee Payroll Management System is a console-based C++ application designed to maintain basic employee records and calculate salary for different employment categories. Payroll processing requires accurate employee information and a consistent method for calculating earnings. Manual calculations can be repetitive and may lead to arithmetic or record-keeping errors.

The proposed application uses object-oriented programming to model employees. An abstract Employee base class defines common employee information and a virtual salary-calculation interface. FullTimeEmployee calculates net salary from basic salary, allowance and deduction, while ContractEmployee calculates payment from hours worked and hourly rate. The PayrollSystem class manages employee objects, supports adding and searching records, displays employees, generates a payroll summary and stores records in a text file.

The project demonstrates encapsulation, inheritance, abstraction and runtime polymorphism through a practical C++17 program. It is an academic prototype and can be extended with database storage, tax rules, attendance integration, payslip generation and secure employee authentication.

TABLE OF CONTENTS

No.	Section
1	Introduction and Background
2	Problem Statement and Proposed Solution
3	Goals and Objectives
4	Scope and Limitations
5	Advantages, Disadvantages and Future Benefits
6	Requirements and Feasibility
7	System Analysis and Functional Modules
8	Object-Oriented Design and Class Relationships
9	OOP Concepts Used
10	Program Logic, Salary Rules and Algorithms
11	C++ Implementation and Source Code
12	Testing and Sample Output
13	Security, Reliability and Future Enhancements
14	Conclusion and References
15	Appendix: Compilation and Viva Questions
16	Detailed System Walkthrough
17	Maintenance and Project Quality

1. INTRODUCTION AND BACKGROUND

Every organization needs a dependable way to maintain employee details and calculate compensation. Payroll information may include employee identity, department, employment category, working hours, earnings, allowances and deductions. A structured payroll application can reduce repetitive calculations and make employee records easier to review.

1.1 What is a payroll management system?

A payroll management system is an application that organizes employee information and calculates the amount payable for a defined pay period. The calculation depends on the organization's salary policy. In this academic model, full-time employees receive basic salary plus allowance minus deduction, while contract employees are paid according to hours worked multiplied by an hourly rate.

1.2 Why C++ and OOP?

C++ supports classes, access control, inheritance, virtual functions and dynamic polymorphism. These features help represent different employee categories without duplicating common employee information. A common interface can calculate salary while each derived class supplies its own formula.

1.3 Why this topic was selected

- Payroll is a familiar business process with clear input, processing and output stages.
- Employee categories provide a natural example of inheritance and polymorphism.
- Salary formulas demonstrate how methods can behave differently for different object types.
- File handling introduces basic data persistence.
- The project can be extended into a larger human-resource information system.

2. PROBLEM STATEMENT AND PROPOSED SOLUTION

2.1 Problem statement

In a manual payroll process, employee details and salary calculations may be maintained in spreadsheets or paper records. Re-entering the same details, applying different salary rules and preparing monthly totals can consume time. Errors may occur if allowances, deductions, hours or hourly rates are entered incorrectly.

2.2 Proposed solution

The proposed system stores employees as objects through a common Employee interface. It supports two categories: full-time and contract employees. The system validates duplicate IDs and basic numeric values, calculates salary using the correct derived-class method, displays records, searches by employee ID and generates a payroll summary.

2.3 Existing approach and proposed application

Manual / basic approach	Proposed C++ application
Separate sheets or paper records	Employee objects stored in one collection
Repeated manual salary calculations	Category-specific salary calculation methods
Manual total calculation	Payroll report calculates the total automatically
Repeated employee ID entry	Duplicate ID check before insertion
Records may not persist consistently	Text-file storage for basic persistence

3. GOALS AND OBJECTIVES

3.1 Main goal

To develop a simple, understandable and reusable Employee Payroll Management System using C++ and object-oriented programming principles.

3.2 Specific objectives

- Design an abstract Employee class containing common employee details.

- Create FullTimeEmployee and ContractEmployee derived classes.
- Implement separate salary formulas for each employment category.
- Use runtime polymorphism to calculate and display salaries through a common interface.
- Allow employee registration, display and search by ID.
- Generate a payroll summary and calculate total payroll expense.
- Validate duplicate IDs and reject negative salary or work values.
- Save and load employee records using file handling.

4. SCOPE AND LIMITATIONS

4.1 Scope

The current version focuses on a small organization's basic employee records and salary calculations. It supports full-time and contract categories, employee search, employee listing, payroll summary and local text-file persistence. It is designed as a single-user console application for an academic demonstration.

4.2 Limitations

- The application does not implement country-specific income tax, provident fund or statutory payroll rules.
- It does not include attendance devices, leave management or overtime approval.
- The program uses text files rather than a relational database.
- There is no login, role-based access or audit history.
- The console interface is less convenient than a graphical or web application.
- It does not generate legally compliant payslips or bank-transfer files.

5. ADVANTAGES, DISADVANTAGES AND FUTURE BENEFITS

5.1 Advantages

- **Consistent calculation:** Salary formulas are implemented in dedicated class methods.
- **Reusable design:** Common employee details are defined once in the base class.
- **Extensibility:** New employee categories can be added through derived classes.
- **Faster summary:** Total payroll is calculated automatically from employee objects.
- **Basic validation:** Duplicate IDs and invalid negative numeric values are rejected.
- **Persistence:** Employee records can be restored from a local text file.

5.2 Disadvantages and challenges

- Text-file storage is not ideal for large or concurrent systems.
- Manual data entry can still introduce mistakes if values are incorrect but valid.
- The sample formulas are simplified and do not represent every organization's payroll policy.
- Security controls and detailed audit logs are not included.
- The program requires a C++ compiler and basic command-line familiarity.

5.3 Future benefits

With additional development, the system can help a small organization standardize routine payroll calculations. Integration with a database, attendance system and secure employee portal could reduce repeated data entry, support reporting and provide employees with access to their own payslips.

6. REQUIREMENTS AND FEASIBILITY

6.1 Hardware requirements

Component	Suggested requirement
Processor	Modern dual-core processor or better
RAM	4 GB minimum; 8 GB recommended
Storage	100 MB free for compiler and project files
Display	Standard laptop or desktop display

6.2 Software requirements

Software	Purpose
GCC / Clang / MSVC	C++ compiler
C++17 support	Language standard used in the implementation
VS Code / Code::Blocks / Visual Studio	Code editing and debugging
Windows / macOS / Linux	Operating system

6.3 Feasibility study

- **Technical feasibility:** Uses standard C++17 language features and the Standard Library.
- **Economic feasibility:** GCC and Clang are free, and the project needs no paid infrastructure.
- **Operational feasibility:** A numbered menu makes the main tasks straightforward.
- **Schedule feasibility:** Work can be divided into class design, salary formulas, menu operations, persistence and testing.

6.4 Assumptions

The program assumes one currency unit for all salary values and one payroll period for each calculation. Full-time net salary is basic salary plus allowance minus deduction. Contract salary is hours worked multiplied by hourly rate. Tax, overtime multipliers and statutory deductions are outside the current scope.

7. SYSTEM ANALYSIS AND FUNCTIONAL MODULES

Module	Description
Full-Time Employee Registration	Accepts identity, department, basic salary, allowance and deduction.
Contract Employee Registration	Accepts identity, department, hours worked and hourly rate.
Employee Display	Lists all employees with category and calculated net salary.
Employee Search	Finds an employee by unique employee ID.
Payroll Report	Displays salary per employee and the total payroll amount.
File Management	Loads saved records at startup and writes updated records to employees.txt.
Menu Controller	Accepts a menu choice and invokes the selected PayrollSystem method.

7.1 Input, process and output

Stage	Examples
Input	Employee ID, name, department, salary components, hours and hourly rate
Process	Validate values, create derived objects, calculate salary, total payroll

Stage	Examples
Output	Employee list, search details, salary values and payroll report

7.2 Typical workflow

- Application starts and creates a PayrollSystem object.
- Saved employee records are loaded from employees.txt when available.
- User selects an operation from the menu.
- The system validates entered data and checks for duplicate IDs.
- A full-time or contract object is created and stored.
- Salary is calculated through the object's virtual method.
- The payroll report can display individual salaries and the overall total.
- Updated employee records are saved to the local file.

8. OBJECT-ORIENTED DESIGN AND CLASS RELATIONSHIPS

The design separates employee data from payroll coordination. Employee is an abstract base class. FullTimeEmployee and ContractEmployee inherit the common identity and department fields, then implement their own salary formula. PayrollSystem owns a vector of smart pointers to Employee, allowing objects of different derived types to be managed through one interface.

Class	Responsibility
Employee (abstract)	Common ID, name and department; virtual salary and type interfaces.
FullTimeEmployee	Stores basic salary, allowance and deduction; calculates net salary.
ContractEmployee	Stores hours worked and hourly rate; calculates contract payment.
PayrollSystem	Manages employee objects, validation, search, reports and file storage.
main()	Creates PayrollSystem and runs the menu loop.

8.1 Relationship diagram



Inheritance connects both employee categories to the common Employee interface. PayrollSystem aggregates employee objects and calls their virtual functions. This is an example of runtime polymorphism.

9. OOP CONCEPTS USED

9.1 Class and object

A class is a blueprint that defines data and behaviour. An object is an instance of a class. Employee, FullTimeEmployee, ContractEmployee and PayrollSystem are classes. Each registered worker is represented by an object of one of the derived classes.

```
FullTimeEmployee staff(1001, "Aarav", "IT",
                       35000, 7000, 2500);
ContractEmployee consultant(1002, "Meera", "Design",
                            120, 350);
```

9.2 Encapsulation

Encapsulation combines data and the operations that work on it. The salary components in FullTimeEmployee and ContractEmployee are private, preventing unrelated code from directly changing them. Public methods expose the required operations, such as calculateSalary() and display().

9.3 Inheritance

Inheritance allows a derived class to reuse common features from a base class. Both employee categories inherit id, name and department from Employee. Each derived class adds fields specific to its payment model.

9.4 Abstraction

Abstraction presents a common interface while hiding category-specific implementation details. Employee declares calculateSalary() and getType() as pure virtual functions. Since these functions have no base implementation, Employee is an abstract class and cannot be instantiated directly.

9.5 Polymorphism

Polymorphism allows one base-class interface to work with objects of different derived classes. PayrollSystem stores unique_ptr values. When it calls calculateSalary(), C++ selects the overridden method for the actual object: the full-time formula or the contract formula.

9.6 Constructors, virtual functions and smart pointers

Constructors initialize objects with their employee details and salary inputs. Virtual functions support dynamic dispatch. The virtual destructor allows safe cleanup through a base-class pointer. unique_ptr manages object lifetime automatically and helps avoid manual delete operations.

9.7 File handling

ifstream reads employee records from employees.txt, while ofstream writes the current records. Each line begins with a type marker (F or C), followed by the fields required to reconstruct the appropriate derived object.

10. PROGRAM LOGIC, SALARY RULES AND ALGORITHMS

10.1 Salary calculation rules

Employee type	Formula	Example
Full-time	Basic + Allowance – Deduction	35,000 + 7,000 – 2,500 = 39,500
Contract	Hours Worked × Hourly Rate	120 × 350 = 42,000

The example values are fictional and are included only to demonstrate the formulas. The program does not apply tax or statutory deductions automatically.

10.2 Add employee algorithm

- Read the proposed employee ID.
- Search the existing collection for the same ID.
- If the ID exists, reject the entry.
- Read the employee name and department.
- Read the salary inputs required for the selected employment type.
- Reject negative values and invalid deduction combinations.
- Create the correct derived object and add it to the collection.
- Save the updated employee records.

10.3 Payroll report algorithm

- Initialize total payroll to zero.
- Visit each employee object in the collection.
- Call calculateSalary() through the base-class interface.
- Display the employee ID, name and calculated salary.
- Add the salary to the total.
- Display the final payroll amount and employee count.

10.4 Search algorithm

The search operation checks each stored employee ID until a match is found. If the ID matches, it displays the employee's name, department, employment category and calculated salary. If no match exists after the scan, it displays an Employee not

found message.

Operation	Time complexity	Reason
Display all employees	O(n)	Every employee is displayed once.
Search employee	O(n)	A linear scan checks IDs.
Add employee	O(n)	Duplicate ID validation scans existing records.
Payroll report	O(n)	Each salary is calculated once.

11. C++ IMPLEMENTATION AND SOURCE CODE

The complete C++17 implementation is provided separately as **employee_payroll_management.cpp**. The following excerpts explain the main object-oriented structure used by the application.

11.1 Abstract Employee base class

```
class Employee {
protected:
    int id;
    string name, department;
public:
    Employee(int i, string n, string d)
      : id(i), name(n), department(d) {}
    virtual double calculateSalary() const = 0;
    virtual string getType() const = 0;
    virtual ~Employee() = default;
};
```

11.2 Full-time employee salary formula

```
class FullTimeEmployee : public Employee {
private:
    double basicSalary, allowance, deduction;
public:
    double calculateSalary() const override {
        return basicSalary + allowance - deduction;
    }
};
```

11.3 Contract employee salary formula

```
class ContractEmployee : public Employee {
private:
    double hoursWorked, hourlyRate;
public:
    double calculateSalary() const override {
        return hoursWorked * hourlyRate;
    }
};
```

11.4 Polymorphic payroll calculation

```
double total = 0;
for (const auto &employee : employees) {
    double salary = employee->calculateSalary();
    cout << employee->getName() << ": " << salary << '\\n';
    total += salary;
}
cout << "Total payroll: " << total << '\\n';
```

11.5 Why unique_ptr is used

The vector stores unique_ptr objects. Each pointer owns one derived employee object, while the base type provides a shared interface. This avoids manual memory management and allows the collection to contain both full-time and contract employees.

11.6 Menu operations

```
1. Add Full-Time Employee
2. Add Contract Employee
3. Display All Employees
4. Search Employee
5. Generate Payroll Report
0. Save and Exit
```

The switch statement in main() maps each menu selection to a PayrollSystem method. The menu repeats until the user chooses Save and Exit.

12. TESTING AND SAMPLE OUTPUT

Testing checks whether each operation behaves as expected for normal, boundary and invalid input. The cases below can be repeated by compiling and running the supplied C++ source file.

Test ID	Action	Expected result
T01	Add full-time employee with valid values	Employee record is created and saved.
T02	Add contract employee with valid hours and rate	Contract record is created and saved.
T03	Enter an existing employee ID	Duplicate ID message is displayed.

Test ID	Action	Expected result
T04	Enter negative salary or work values	Input is rejected.
T05	Search for an existing employee ID	Employee details and calculated salary appear.
T06	Search for an unknown ID	Employee not found message appears.
T07	Generate payroll report	Individual salaries and total payroll appear.
T08	Exit and restart the program	Saved employee records are loaded.

12.1 Illustrative console output

```
==========================================
       EMPLOYEE PAYROLL MANAGEMENT SYSTEM
==========================================
1. Add Full-Time Employee
2. Add Contract Employee
3. Display All Employees
4. Search Employee
5. Generate Payroll Report
0. Save and Exit

========== MONTHLY PAYROLL REPORT ==========
1001    Aarav Sharma          39500.00
1002    Meera Kapoor          42000.00
------------------------------------------
Total payroll: 81500.00
Employee count: 2
```

The sample names and amounts are fictional. Actual output depends on the employee records present in employees.txt.

13. SECURITY, RELIABILITY AND FUTURE ENHANCEMENTS

13.1 Reliability

The program checks for duplicate employee IDs and validates important numeric values before creating records. It also uses a virtual destructor and smart pointers to manage object lifetimes. A production version should additionally validate every input stream operation and report file read/write failures.

13.2 Security and privacy

- Add authenticated accounts for payroll administrators.
- Restrict salary information to authorized roles.
- Encrypt backups and protect employee personal information.
- Maintain an audit log for changes to salary components.
- Avoid storing sensitive bank or identity information in plain text.

13.3 Future enhancements

- **Database connectivity:** Replace text files with MySQL or PostgreSQL.
- **Tax and deductions:** Add configurable tax, provident fund and insurance rules based on applicable policy.
- **Attendance integration:** Calculate payable days and approved overtime from attendance records.
- **Payslip generation:** Produce printable PDF payslips for each employee.
- **Graphical interface:** Add a desktop or web interface for HR staff.
- **Employee portal:** Allow staff to securely view their own payslips and payroll history.
- **Reporting:** Generate department-wise salary summaries and monthly expense trends.
- **Cloud backup:** Maintain encrypted backups and controlled access.

14. CONCLUSION AND REFERENCES

14.1 Conclusion

The Employee Payroll Management System demonstrates how object-oriented programming can model a real business process. A common Employee interface is extended by FullTimeEmployee and ContractEmployee, each with its own salary calculation. PayrollSystem manages the objects, validates entries, searches employee records, displays information and

generates a total payroll report.

The project demonstrates classes, objects, encapsulation, inheritance, abstraction, runtime polymorphism, constructors, virtual destructors, smart pointers, vectors and file handling. It provides a foundation for a more advanced payroll application with database storage, secure access and organization-specific payroll rules.

14.2 Learning outcomes

- Designing abstract classes and derived classes.
- Applying the four major OOP principles in C++.
- Using virtual functions for runtime polymorphism.
- Managing dynamic objects using unique_ptr.
- Implementing salary formulas and validation logic.
- Using vectors and file streams from the C++ Standard Library.
- Preparing test cases and identifying future improvements.

14.3 References

- ISO C++ — https://isocpp.org/
- cppreference — https://en.cppreference.com/
- cplusplus.com — https://cplusplus.com/doc/tutorial/
- Bjarne Stroustrup, The C++ Programming Language.
- Deitel and Deitel, C++ How to Program.

15. APPENDIX: COMPILATION AND VIVA QUESTIONS

15.1 Compilation steps

- Install GCC, Clang or another C++17-compatible compiler.
- Save the source as employee_payroll_management.cpp.
- Open a terminal in the source folder.
- Compile using the command below.
- Run the executable and test each menu option.
- Keep employees.txt in the program's working directory for saved records.

```
g++ -std=c++17 employee_payroll_management.cpp -o payroll
./payroll
```

15.2 Important project files

File	Purpose
employee_payroll_management.cpp	Complete C++ classes, methods and main menu.
employees.txt	Stores full-time and contract employee records.
README.txt	Contains student details and compilation instructions.

15.3 Key OOP terms

Term	Meaning
Class	Blueprint defining data and behaviour.
Object	An instance of a class.
Encapsulation	Combining data and methods while controlling access.
Inheritance	Deriving a new class from an existing base class.
Abstraction	Exposing an interface while hiding implementation details.
Polymorphism	One interface supporting different implementations.
Virtual function	A function supporting dynamic dispatch.
Pure virtual function	A virtual function declared with = 0.
Smart pointer	An object that manages a dynamically allocated resource.

15.4 Viva questions and answers

- **What is OOP?** A programming approach that organizes software around objects containing data and behaviour.
- **Why is Employee abstract?** It contains pure virtual functions and represents a common interface.
- **Which classes inherit Employee?** FullTimeEmployee and ContractEmployee.
- **How is polymorphism used?** The system calls calculateSalary() through an Employee pointer.
- **What is the full-time salary formula?** Basic salary plus allowance minus deduction.
- **What is the contract salary formula?** Hours worked multiplied by hourly rate.
- **Why use a virtual destructor?** To support correct cleanup through a base-class pointer.
- **What is unique_ptr?** A smart pointer that provides exclusive ownership of an object.

16. DETAILED SYSTEM WALKTHROUGH

16.1 Application startup

When main() creates a PayrollSystem object, the constructor calls loadData(). The program reads employees.txt line by line. A type marker identifies whether the record represents a full-time or contract employee. The relevant derived object is reconstructed and added to the collection. If no records are loaded, sample records are inserted for demonstration.

16.2 Adding a full-time employee

The application asks for an employee ID and checks whether it already exists. It then collects the employee name, department, basic salary, allowance and deduction. The program rejects negative components and prevents a deduction greater than the sum of basic salary and allowance. If the values are acceptable, it creates a FullTimeEmployee object and saves the updated collection.

```
Read employee ID
       |
Check for duplicate ID
       |
Read name, department and pay values
       |
Validate numeric values
       |
Create FullTimeEmployee object
       |
Store object and save file
```

16.3 Adding a contract employee

The contract registration operation collects the employee ID, name, department, hours worked and hourly rate. It checks for a duplicate ID and rejects negative hours or rates. A ContractEmployee object is then created. Its salary is calculated by multiplying hours worked by the hourly rate.

16.4 Generating a payroll report

The report method starts with a total of zero and loops over the employee collection. For every employee it calls calculateSalary() through the base-class pointer. The correct derived-class formula runs automatically. Each result is displayed and added to the overall payroll amount.

16.5 Searching for an employee

The search operation reads an employee ID and compares it with the ID of each object. If a match is found, the application displays the name, department, employee category and calculated salary. Otherwise, it reports that the employee was not found.

16.6 Saving and loading

The serialize() method of each derived class writes a type marker and the relevant fields. The loadData() method reads that marker and constructs the matching class. This demonstrates basic persistence, although a production application should use robust serialization or a database.

17. MAINTENANCE, QUALITY AND PROJECT EVALUATION

17.1 Suggested database structure for an advanced version

Table	Possible columns
Employee	employee_id, name, department_id, employment_type, joining_date
FullTimePay	employee_id, basic_salary, allowance, deduction
ContractPay	employee_id, hourly_rate
WorkLog	worklog_id, employee_id, work_date, hours
PayrollRun	payroll_id, period_start, period_end, generated_at
PayrollItem	payroll_id, employee_id, gross_pay, deductions, net_pay

17.2 Maintenance plan

- Back up employee data before changing the file format.
- Keep employee IDs unique and stable over time.
- Test salary calculations after changing any payroll formula.
- Compile with warnings enabled and review compiler messages.
- Add a test case whenever a new menu feature is introduced.
- Document assumptions about pay periods, deductions and rounding.

17.3 Quality attributes

Usability: A numbered menu supports basic navigation. **Maintainability:** Separate classes divide employee categories and payroll management. **Extensibility:** New employee categories can implement the Employee interface. **Portability:** Standard C++17 features support compilation on common operating systems. **Reliability:** Duplicate ID and numeric checks reduce common data-entry errors.

17.4 Known limitations and responsible use

This program is an academic prototype, not a complete payroll product. It does not implement legal tax calculations, statutory compliance, bank payment, secure authentication, concurrent users or a complete audit trail. Real payroll software must follow the organization's applicable legal and accounting requirements and protect employee data.

17.5 Final submission checklist

- Confirm student name, UID, section, teacher and subject code on the cover page.
- Compile the source using the C++17 standard.
- Test both employee categories and each salary formula.
- Test duplicate IDs and invalid negative values.
- Generate a payroll report and verify the total manually.
- Keep the PDF report and C++ source file together for submission.

— END OF PROJECT REPORT —